Mandatory 1 - Anvendt Kryptografi

Mads Lindersson - Mali0002

Valg af opgave: Clients - server løsning

Hvordan bruger du applikationen

Stå i mappen mads_lindersson_mandatory_1.
Kør server.py med kommandoen: *python server.py*.
Åben en ny terminal.
Kør client.py med kommandoen: *python client.py*.
Følg instruktionerne i terminalen.
Beskeder fra client1 vises på serverens terminal.

Kryptografisk Implementation

Der andvendes KDF'en Bcrypt til at udføre hashing i applikationen. Bcrypt er utroligt godt til hashing af passwords da den er meget svær at bryde, og derfor ikke feasible for en ondsindet part. Så længe der bliver brugt 8 characters og minimum 3 klasser af tegn på et password, så vil det tage for lang tid at brute force Bcrypt hashes.
Source: https://specopssoft.com/blog/hashing-algorithm-cracking-bcrypt-passwords/

Hashet gemmes hardcoded i en dictionary på serveren, så altså gemmes det in memory og er ikke særligt sikkert.

```
accounts = {"client1": '$2b$16$u6ICB8a/Iu8WuXvacZMXq.EVbxWnhM/SJCjHOXstjw4sB4VJsNwa6'}
```

Men da der var en begrænsning på antal af filer der måtte laves, var dette en nødvendig løsning.

Der krypteres ved brug af en udvidet version af Caesar Cipher hvor der også kan bruges tal og symboler. Helt konkret bruges Ascii tabellen. Dette er ikke den mest sikre måde at kryptere på, men da vi ikke har lært moderne kryptering endnu, er det bedre end at sende i plaintext.

```
76  def encrypt_message(msg, shift=3):
77      result = ""
78      for char in msg:
79          ascii_num = ord(char)
80          if 32 <= ascii_num <= 126:
81              result += chr((ascii_num - 32 + shift) % 95 + 32)
82          else:
83              result += char
84      return result
```

```
86  def decrypt_message(msg, shift=3):
87      result = ""
88      for char in msg:
89          ascii_num = ord(char)
90          if 32 <= ascii_num <= 126:
91              result += chr((ascii_num - 32 - shift) % 95 + 32)
92          else:
93              result += char
94      return result
```

Både klienten og serveren kan kryptere og dekryptere sådan at de kan kommunikere uden at sende noget i plaintext. Serveren sender sådan set ikke andet end f.eks. “LOGIN_OK” men hvis en ondsindet person ser dette, så ved de også præcis hvor login sker henne, og der er ingen grund til at gøre det nemmere for personen at gøre noget ondsindet.

Der bruges en “key” eller en “shift” som kommer fra Caesar Cipher. Denne er sat som default på både klienten og serveren til at være 3. Der bliver brugt shift = 3 ved oprettelse af forbindelse mellem klienten og serveren. Når serveren sender en “LOGIN_OK” besked til klienten efter et korrekt login, sender serveren en shift værdi med og klienten bruger nu denne shift til at kryptere med, dvs. At serveren bestemmer hvilken shift som der bliver kommunikeret med.

```
27              msg = {
28                  "status": "LOGIN_OK",
29                  "shift": shift
30              }
31
32              json_msg = json.dumps(msg)
33              msg_encrypted = encrypt_message(json_msg)
34              clientsocket.sendall(msg_encrypted.encode("utf-8"))
```

Dette øger sikkerheden yderligere, men hvis en ondsindet person finder ud af at første kommunikation sker med en shift = 3, så kan personen hurtigt finde ud af hvilken shift der kommunikeres med efterfølgende.

Wireshark capture

Det kan ses i Wireshark at der sker et TCP handshake.

7	2.418776300	127.0.0.1	127.0.0.1	TCP	56 58410 → 12001 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK…
8	2.418840400	127.0.0.1	127.0.0.1	TCP	56 12001 → 58410 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 …
9	2.418860600	127.0.0.1	127.0.0.1	TCP	44 58410 → 12001 [ACK] Seq=1 Ack=1 Win=65280 Len=0

Efterfølgende sender klienten dens username og password og serveren svarer tilbage med “LOGIN_OK” og en shift = 5 hvorpå klienten så skriver en besked, beskeden her er “Hej_1”. Alt bliver krypteret så det kan ikke læses med det blotte øje

